

Dokumentacja Zadania 1:

Algorytm szyfrowania afinicznego z mechaniczną obfuskacją kluczy

Autor: Mateusz Grabowski (mg315933@student.polsl.pl)

Kierunek: Informatyka (profil praktyczny), Wydział Matematyki Stosowanej, Politechnika Śląska

Specjalizacja: Bezpieczeństwo Systemów i Aplikacji Sieciowych

Język implementacji: C++

1. Założenia projektu i środowisko uruchomieniowe

Projekt stanowi implementację klasycznego szyfru afinicznego przystosowanego do operowania na surowych strumieniach danych (binarnych plikach dowolnego formatu). Ze względu na bajtowy charakter operacji, wielkość przestrzeni znaków n wynosi 256.

Aplikacja została zaprojektowana zgodnie z dobrymi praktykami tworzenia narzędzi konsolowych. Ścieżki do przetwarzanych plików przyjmowane są dynamicznie przez argumenty wywołania programu (`argv`), co ułatwia ewentualną integrację ze skryptami testowymi.

2. Model matematyczny i generacja kluczy (Entropia)

W celu zminimalizowania ryzyka błędu ludzkiego oraz podniesienia niezawodności narzędzia, zrezygnowano z ręcznego podawania kluczy szyfrujących na rzecz ich automatycznej generacji.

1. **Źródło losowości:** Klucze a (mnożnik) oraz b (przesunięcie) wyliczane są na podstawie czasu systemowego (Unix Timestamp), co zapewnia unikalność szyfrowania przy każdym wywołaniu.
2. **Warunek odwracalności:** Aby istniała funkcja deszyfrująca, klucz a oraz rozmiar alfabetu n muszą być względnie pierwsze ($NWD(a, 256) = 1$). Aplikacja sprawdza parzystość wygenerowanego klucza a i w razie

potrzeby inkrementuje go o 1. Gwarantuje to matematyczną odwracalność szyfrogramu.

3. **Funkcja szyfrująca:** Każdy bajt jest modyfikowany zgodnie ze wzorem:

$$E(x) = (a * x + b + 256) \pmod{256}$$

Dodanie wartości 256 przed operacją modulo zabezpiecza program przed nieprzewidywalnym zachowaniem operatora reszty z dzielenia w języku C++ dla wartości ujemnych.

3. Architektura autorskiego formatu **.aff1**

Standardowe systemy kryptograficzne przechowują klucze w wyrażnie oddzielonych, ustandaryzowanych nagłówkach. W tym projekcie zaimplementowano mechanizm obfuskacji przestrzennej. Surowe klucze a i b ukrywane są (wplatane) wewnątrz zaszyfrowanego ładunku – odpowiednio w połowie i w trzech czwartych długości bufora.

Offset	Rozmiar	Zawartość	Rola w systemie
0x00	4 bajty	AFF1	Sygnatura weryfikująca format przed deszyfracją.
Zmienny	Zmienny	lewa_czesc	Zaszyfrowane bajty do połowy zdefiniowanego podziału.
Zmienny	1 bajt	Klucz a	Wpleciony w strumień danych klucz mnożnika.
Zmienny	Zmienny	srodek	Zaszyfrowane bajty między kluczem \$a\$ a \$b\$.
Zmienny	1 bajt	Klucz b	Wpleciony w strumień danych klucz przesunięcia.
Zmienny	Zmienny	prawa_czesc	Ostatnia partia zaszyfrowanych bajtów pliku.
EOF - 14	4 bajty	EOF\0	Znacznik końca ładunku i początku bloku metadanych.
EOF - 10	4 bajty	pozycja_A	Bezwzględny offset klucza a.
EOF - 6	2 bajty	WB	Zabezpieczenie integralności znaczników.
EOF - 4	4 bajty	pozycja_B	Bezwzględny offset klucza b.

4. Przepływ sterowania (Control Flow)

- **Proces szyfrowania:** Plik ładowany jest do pamięci operacyjnej znak po znaku w trybie binarnym. Po wykonaniu matematycznego przesunięcia afinicznego, wskaźniki insercji kluczy ustalane są na podstawie zrzutowania wymiarów bufora. Struktura jest następnie serializowana na dysk z zachowaniem obfuskacji i dodaniem rozszerzenia `.aff1`.
- **Proces deszyfrowania:** Proces uodporniono na uszkodzenia nagłówka. Po poprawnej walidacji sygnatury początkowej, program skanuje strumień pamięci wstecz (od EOF) w poszukiwaniu unikalnego ciągu kończącego blok danych `EOF\0`. Po jego zlokalizowaniu następuje ekstrakcja wskaźników do ukrytych kluczy a i b . Odtwarzanie pliku bazuje na wyliczeniu elementu odwrotnego a^{-1} algorytmem brutalnego przeszukiwania (dopuszczalne dla małego m) i oryginalnych bajtów za pomocą funkcji:

$$D(x) = a^{-1} * (x - b + 256) \pmod{256}$$

Podczas rekonstrukcji algorytm aktywnie nasłuchuje offsetów, selektywnie pomijając bajty pełniące rolę ukrytych kluczy, a finalny plik zapisywany jest z sufiksem `_odszyfrowany`.

5. Kluczowe fragmenty implementacji

W tej sekcji przedstawiono wybrane fragmenty kodu źródłowego, które realizują najważniejsze założenia architektoniczne i kryptograficzne systemu. Pełny kod źródłowy znajduje się w załączonym repozytorium.

5.1. Dynamiczna generacja i walidacja kluczy

Zamiast statycznego podawania kluczy przez użytkownika, program wykorzystuje czas systemowy (Unix Timestamp) jako źródło entropii do wygenerowania pary kluczy a i b .

```
//zmienne potrzebne do szyfrowania
long int t = static_cast<long int>(time(NULL)); //czas systemu według unix
int n = 256;
//wyliczamy a i b na podstawie czasu systemowego
int a = t % n;
if (a % 2 == 0) a += 1; //upewniamy się, że liczba nie jest parzysta (co uniemożliwiłoby odszyfrowanie)
int b = (t / n) % n;
```

Najważniejszą linią w tym bloku jest instrukcja warunkowa sprawdzająca parzystość wyliczonego mnożnika a . Ponieważ operujemy w pierścieniu Z_{256} , klucz a i wartość 256 muszą być względnie pierwsze ($\text{NWD}(a, 256) = 1$). Automatyczna inkrementacja parzystej wartości o 1 jest zabezpieczeniem przed nieodwracalną utratą danych. Gwarantuje ona, że odwrotność modularna zawsze będzie istnieć.

5.2. Mechanizm wpłatania kluczy i zapis

Proces ukrywania wygenerowanych kluczy bezpośrednio w strumieniu zaszyfrowanych danych, zamiast w standardowym nagłówku pliku i zapis danych do pliku w formacie .aff1

```

//obliczamy gdzie umiescic a i b
int rozmiar = bufor.length();
int pozycja_A = t % (rozmiar / 2 + 1); //upewniamy sie, ze
pozycja wsp. A bedzie w pierwszej polowie pliku
int pozycja_B = pozycja_A + (rozmiar / 4); //pozycja wsp. B
po A (ale nie dalej niz koniec pliku)

//dzielimy zaszyfrowany tekst na: START+A+MIDDLE+B+END
string lewa_czesc = bufor.substr(0, pozycja_A);
string srodek = bufor.substr(pozycja_A, pozycja_B - pozycja
_A);
string prawa_czesc = bufor.substr(pozycja_B);

[...]

//zapis do pliku .AFF1
char ciag_AFF1[] = {'A', 'F', 'F', '1'};
plik_wyjsciowy.write(ciag_AFF1, sizeof(ciag_AFF1));

plik_wyjsciowy << lewa_czesc;
plik_wyjsciowy << (char)a;
plik_wyjsciowy << srodek;
plik_wyjsciowy << (char)b;
plik_wyjsciowy << prawa_czesc;
//Zaszyfrowana czesc z wspolczynnikami zapisana, zapisujemy
nasz koniec pliku

char ciag_EOF[] = {'E', 'O', 'F', '\0'};
plik_wyjsciowy.write(ciag_EOF, sizeof(ciag_EOF));
plik_wyjsciowy.write(reinterpret_cast<const char*>(&pozycja
_A), sizeof(pozycja_A)); //zapisujemy surowe 4 bajty wspolc
zynnika A do pliku.
char ciag_WB[] = {'W', 'B'};
plik_wyjsciowy.write(ciag_WB, sizeof(ciag_WB));
plik_wyjsciowy.write(reinterpret_cast<const char*>(&pozycja
_B), sizeof(pozycja_B)); //zapisujemy surowe 4 bajty wspolc
zynnika B do pliku.

```

Program dynamicznie dzieli główny bufor danych na trzy sekcje, a następnie wkleja pomiędzy nie pojedyncze bajty kluczy (`(char)a` oraz `(char)b`). Miejsca wstrzyknięcia kluczy są pseudolosowe (uzależnione od czasu `t`), co znacząco utrudnia potencjalną kryptoanalizę strukturalną pliku przez osoby trzecie.

5.3. Deszyfrowanie i filtrowanie strumienia w locie

Podczas odtwarzania pliku algorytm musi odseparować ukryte klucze od właściwych danych, aby uniknąć korupcji pliku wynikowego.

```
while (plik.get(znak) && plik.tellg() <= pozycjaEOF) { //do
    poki nie dojdziemy do EOF\0 pobieramy jeden znak
        if (plik.tellg() == odczytana_pozycja_A + 1 || plik.te
            llg() == odczytana_pozycja_B + 1) { //pomijamy pozycje A i
                B
                    continue;
            }
            char odszyfrowany_znak = (a_odwrotne * (znak - b + n))
            % n; //odwrotna funkcja do funkcji szyfrującej
            bufor = bufor + odszyfrowany_znak;
        }
```

Zamiast ładować cały plik do bufora i usuwać z niego klucze, program iteruje po strumieniu wejściowym odczytując aktualną pozycję wskaźnika (`tellg()`). Gdy wskaźnik strumienia zrówna się z zapisaną wcześniej pozycją klucza, iteracja jest przerywana (`continue`). Zapewnia to płynną ekstrakcję oryginalnych bajtów bez dodatkowego narzutu na pamięć.

6. Testy i weryfikacja integralności danych

W celu potwierdzenia niezawodności implementacji i braku strat na poziomie binarnym, przeprowadzono zestaw testów walidacyjnych, pokrywających zarówno standardowe, jak i skrajne wektory użycia aplikacji.

Do weryfikacji użyto standardowej funkcji skrótu **SHA-256**. Identyczność hashów przed zaszyfrowaniem i po deszyfracji stanowi kryptograficzny dowód na bezstratność operacji.

Wektor testowy 1: Przypadek graniczny (Plik 1-bajtowy)

- **Cel testu:** Sprawdzenie stabilności arytmetyki wskaźników bufora dla mikroskopijnych plików, gdzie potencjalnie mogłoby dojść do kolizji (nakładania się) wyliczanych offsetów dla pozycji A i B.

- **Dane wejściowe:** Plik `test.txt` o zawartości dokładnie jednego znaku ASCII.
- **Wynik:** Aplikacja poprawnie wyliczyła proporcjonalne pozycje, umieściła klucze w buforze, zachowała strukturę nagłówków `AFF1` oraz metadanych `EOF\0`, nie powodując błędów naruszenia ochrony pamięci. Deszyfracja przywróciła pierwotny, 1-bajtowy plik.

```
> hexdump -C test.txt
00000000 61                                     |a|
00000001

Semestr 5/CyberPrakt/Zadanie 1 via Δ
> ./szyfrowanie test.txt
Obliczone współczynniki:
  t = 1791456043
  a = 43
  b = 115
Obliczona pozycja współczynników:
  poz_A = 0
  poz_B = 1
Zaszyfrowany plik zapisany poprawnie.

Semestr 5/CyberPrakt/Zadanie 1 via Δ
> hexdump -C test.txt.aff1
00000000 41 46 46 31 2b 73 be 45 4f 46 00 00 00 00 57 |AFF1+s.EOF....W|
00000010 42 01 00 00 00
00000015

Semestr 5/CyberPrakt/Zadanie 1 via Δ
> ./odszyfrowanie test.txt.aff1
Poprawny nagłówek pliku!
Znaleziono znacznik EOF
Obecna pozycja znacznika odczytu pliku: 11
Oczytana pozycja parametru A: 0
Poprawny ciąg WB
Oczytana pozycja parametru B: 1
Odczytane z pliku wartości:
  a = 43
  b = 115
Zaszyfrowany plik zapisany poprawnie.

Semestr 5/CyberPrakt/Zadanie 1 via Δ
> hexdump -C test_odszyfrowany.txt
00000000 61                                     |a|
00000001
```

Plik tekstowy o długości 1 bajta poprawnie zaszyfrowany i odszyfrowany. W hexdump zaszyfrowanego pliku widać nagłówek `AFF1`, `EOF\0`, 4 bajty pozycji A, WB, 4 bajty pozycji B.

```
Semestr 5/CyberPrakt/Zadanie 1 via Δ
> sha256 ./test.txt
SHA256 (./test.txt) = ca978112ca1bbdcaf231b39a23dc4da786eff8147c4e72b9807785afee48bb

Semestr 5/CyberPrakt/Zadanie 1 via Δ
> sha256 ./test_odszyfrowany.txt
SHA256 (./test_odszyfrowany.txt) = ca978112ca1bbdcaf231b39a23dc4da786eff8147c4e72b9807785afee48bb
```

SHA256 identyczne dla obu plików.

Wektor testowy 2: Strumień binarny (Grafika 1.5 MB)

- **Cel testu:** Udowodnienie, że program operuje na surowych bajtach (`std::ios::binary`) i nie jest podatny na interpretację znaków specjalnych (np. bajtów `0x00`, znaków nowej linii) jako końca łańcucha znaków, a struktura wyszukiwania znacznika `EOF\0` od tyłu zadziała poprawnie przy dużej ilości losowych danych.
- **Dane wejściowe:** Plik graficzny `zdjecie.jpg` o rozmiarze 1.5 MB.
- **Wynik (Protokół integralności):**

- Hash **SHA-256** oryginału:
4dc0b022b42db5fb8ea8b14eaf63914664cdf7984998d81aba2153926fdff553
- Hash **SHA-256** po deszyfracji:
4dc0b022b42db5fb8ea8b14eaf63914664cdf7984998d81aba2153926fdff553
- Grafika po odszyfrowaniu ładuje się prawidłowo w przeglądarce obrazów.
Sumy kontrolne pokrywają się w 100%, co wyklucza jakąkolwiek korupcję bajtów lub przesunięcia bufora w algorytmie obfuskacji.

```
Semestr 5/CyberPrakt/Zadanie 1 via ^
> ./szyfrowanie zdjecie.jpg
Obliczone współczynniki:
  t = 1791456331
  a = 75
  b = 116
Obliczona pozycja współczynnika:
  poz_A = 284949
  poz_B = 443685
Zaszyfrowany plik zapisany poprawnie.

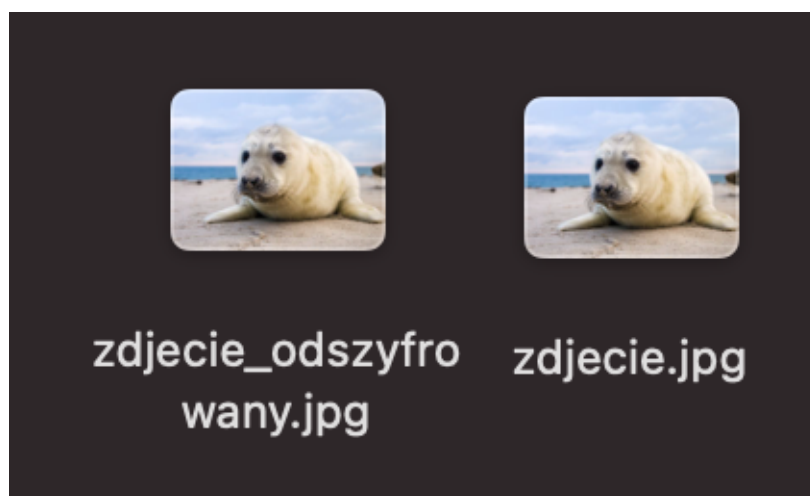
Semestr 5/CyberPrakt/Zadanie 1 via ^ took 3s
> ./odszyfrowanie zdjecie.jpg.aff1
Poprawny nagłówek pliku!
Znaleziono znacznik EOF
Obecna pozycja znacznika odczytu pliku: 634950
Oczytana pozycja parametru A: 284949
Poprawny ciąg WB
Oczytana pozycja parametru B: 443685
Odczytane z pliku wartości:
  a = 75
  b = 116
Zaszyfrowany plik zapisany poprawnie.

Semestr 5/CyberPrakt/Zadanie 1 via ^ took 4s
> sha256 ./zdjecie.jpg
SHA256 (./zdjecie.jpg) = 4dc0b022b42db5fb8ea8b14eaf63914664cdf7984998d81aba2153926fdff553

Semestr 5/CyberPrakt/Zadanie 1 via ^
> sha256 ./zdjecie_odszyfrowany.jpg
SHA256 (./zdjecie_odszyfrowany.jpg) = 4dc0b022b42db5fb8ea8b14eaf63914664cdf7984998d81aba2153926fdff553

Semestr 5/CyberPrakt/Zadanie 1 via ^
>
```

Po zaszyfrowaniu i odszyfrowaniu checksum SHA256 jest identyczny.



Zdjęcie poprawnie wyświetla się w przeglądarce obrazów.